

Hanoi University of Science and Technology
School of Information and Communication Technology

Course Project Report

Multiscale and Multilayer Contrastive Learning
for Domain Generalization

Reproduction, Implementation Analysis, and Experimental Discussion

Group: 26
Students: Vu Thuong Tin
Chu Anh Duc
Nguyen Xuan Khai
Project topic: Domain Generalization in Computer Vision
Main method: M^2 -CL

June 10, 2026

Contents

1	Introduction	1
1.1	Project Motivation	1
1.2	Report Scope	2
1.3	Main Contributions of This Report	2
2	Problem Formulation	2
3	Related Work	3
4	Proposed Method	3
4.1	Backbone Architecture	3
4.2	Extraction Block	4
4.3	Contrastive Regularization	5
5	Implementation in the Project Codebase	5
5.1	Repository Organization	5
5.2	Model Construction	6
5.3	Extraction Block Details	6
5.4	Loss Implementation	7
5.5	Training Pipeline	7
5.6	Configuration Defaults	7
6	Reproduction Workflow	8
7	Experimental Setup	8
7.1	Datasets	8
7.2	Training Configuration	8
7.3	Baselines	9
8	Quantitative Results	9
8.1	Result Provenance and Reliability	10
8.2	Per-Dataset Interpretation	10
8.3	What the Numbers Suggest About the Method	10
9	Ablation and Sensitivity Analysis	11
9.1	Design Lessons from the Ablation	11
10	Qualitative Results	12
10.1	How to Read the Saliency Maps	12
11	Discussion	13
11.1	Comparison with the Original Paper	13
11.2	Practical Engineering Observations	13
11.3	Threats to Validity	14
12	Course Project Reflection	14
13	Limitations	14
14	Conclusion	15
A	Full Result Tables	15

Abstract

Domain generalization (DG) studies the problem of training a visual recognition model on a set of source domains and deploying it on unseen target domains without access to target-domain samples during training. This setting is difficult because convolutional neural networks often exploit domain-specific shortcuts such as background, texture, rendering style, or acquisition bias instead of learning class-causal visual attributes. This report presents a course-project reproduction and analysis of M^2 -CL, a multiscale and multilayer contrastive learning framework for DG. Unlike the original research paper, this document is written as a university project report: it explains the motivation, connects the mathematical idea to the implementation in the submitted codebase, describes the training pipeline, discusses the obtained and placeholder results separately, and reflects on practical reproducibility issues. The method extends a standard ImageNet-pretrained ResNet backbone with extraction blocks attached to intermediate convolutional layers. These blocks compress and pool multiscale feature maps into a representation that combines low-level and high-level evidence. A supervised contrastive regularizer is then applied to intermediate representations so that samples of the same class become closer while samples of different classes become more separable. We evaluate the method on PACS, VLCS, and Office-Home using leave-one-domain-out evaluation, compare it against representative DG baselines, and analyze both quantitative accuracy and qualitative saliency-map behavior.

1 Introduction

Modern image classifiers achieve strong in-distribution performance, but their accuracy can drop sharply when the test images come from a visual context that was absent during training. This is common in practical applications: a model trained on photos may be evaluated on sketches, artworks, cartoons, medical images from another device, or objects captured in a different environment. The central problem is that empirical risk minimization (ERM) does not distinguish between class-causal evidence and domain-specific shortcuts. If a training domain repeatedly associates a class with a background, texture, or color distribution, a high-capacity CNN can learn that correlation and still achieve low source-domain error.

Domain generalization addresses this failure mode under a strict constraint: target-domain images and labels are unavailable during training. A DG model must therefore learn representations that remain useful under distribution shift by relying only on the diversity present in source domains. The paper *Multiscale and Multilayer Contrastive Learning for Domain Generalization* by Ballas and Diou proposes that intermediate CNN features contain useful information for separating invariant class attributes from domain-specific artifacts. Instead of using only the last feature vector of a ResNet, the method extracts representations from multiple layers and multiple spatial scales.

This project implements and analyzes the core idea of that paper. The presentation slides focus on three standard DG benchmarks used in the original work: PACS, VLCS, and Office-Home. NICO is discussed in the original paper, but it is outside the experimental scope of the current repository and report.

1.1 Project Motivation

The motivation of this project is not only to restate the paper but also to understand whether the method can be reconstructed as a working training pipeline. This distinction matters in a course project. A research paper normally emphasizes novelty, mathematical formulation, and final benchmark numbers. A project report should additionally answer more practical questions:

- What exact problem is being solved, and why is the usual supervised-learning setting insufficient?

- Which part of the published method is implemented in code, and which parts are simplified or left out?
- How are datasets, domains, training splits, checkpoints, and evaluation metrics organized?
- Do the results support the intended conclusion, and where are the uncertainties?
- What did the implementation reveal that is not obvious from the paper alone?

For that reason, this report treats M^2 -CL as both a computer-vision method and a reproducibility exercise. The method is studied from three angles: the conceptual DG problem, the architecture and loss design, and the concrete PyTorch implementation in the submitted repository.

1.2 Report Scope

The original paper evaluates on PACS, VLCS, Office-Home, and NICO. The current repository contains dataset loaders for all four datasets, but the report focuses mainly on PACS, VLCS, and Office-Home because those are the datasets used in the project tables and slides. NICO is mentioned when discussing qualitative saliency examples and future extensions, but it is not treated as a completed quantitative reproduction.

The implementation supports more methods than are deeply analyzed in the main text. The comparison tables include ERM, RSC, CORAL, Mixup, MMD, SagNet, SelfReg, ARM, EQRM, SAGM, M^2 , and M^2 -CL. However, the main technical discussion focuses on the difference between ERM, M^2 , and M^2 -CL, because these three models isolate the two core ideas of the project: multilayer feature extraction and contrastive regularization.

1.3 Main Contributions of This Report

This project report makes the following contributions relative to the original paper:

1. It explains the paper’s DG setting in a course-report style, with explicit source-domain and target-domain notation.
2. It maps the proposed architecture to the local model, extraction-block, loss, and training modules.
3. It summarizes the implemented training and evaluation workflow, including dataset construction, source validation holdout, checkpoint selection, and result aggregation.
4. It separates real experimental outputs from one-seed estimates and synthetic placeholders, so the reader can judge the strength of the empirical evidence correctly.
5. It adds practical observations about memory usage, batch composition, and reproducibility that are important when turning a paper into code.

2 Problem Formulation

Let $\mathcal{D} = \{D_1, D_2, \dots, D_N\}$ be a set of source domains. Each domain contains image-label pairs (x, y) sampled from a domain-specific joint distribution over $\mathcal{X} \times \mathcal{Y}$. In the leave-one-domain-out protocol, one domain is selected as the unseen target domain, while the remaining domains are used for training and validation. The goal is to learn a classifier

$$f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$$

that minimizes target-domain error despite never observing target-domain data during training.

The key challenge is representation entanglement. A latent representation may contain both class-specific attributes, such as object shape and discriminative parts, and domain-specific attributes, such as sketch strokes, cartoon color palettes, background scenes, lighting, or image texture. If the classifier relies heavily on the latter, it may fail when the target domain changes. The desired behavior is therefore to produce features that preserve class information while suppressing unstable domain-specific correlations.

3 Related Work

Several families of methods have been proposed for DG. ERM is the simplest baseline and trains a classifier by minimizing cross-entropy over pooled source-domain data. Regularization methods such as RSC suppress dominant features during training to prevent the model from relying on narrow shortcuts. Distribution alignment methods such as CORAL and MMD encourage feature distributions from different domains to become closer. Data-mixing methods such as Mixup construct interpolated training samples to improve smoothness and robustness. Style-oriented methods such as SagNet attempt to reduce dependence on domain-specific style. Other methods, including ARM, EQRM, SelfReg, and SAGM, introduce meta-learning, risk distribution, self-supervised regularization, or sharpness-aware training mechanisms.

M^2 -CL differs from these approaches by explicitly reusing intermediate convolutional features and regularizing them with a supervised contrastive objective. This design is motivated by the idea that early and middle CNN layers can still contain spatial and local visual evidence that may be lost or entangled by the final global representation.

Table 1: High-level comparison of DG method families used in the project.

Family	Representative methods	Main idea
Standard supervision	ERM	Pool all source domains and minimize cross-entropy.
Feature suppression	RSC	Hide highly predictive features to reduce shortcut reliance.
Distribution alignment	CORAL, MMD	Penalize differences between source-domain feature distributions.
Data interpolation	Mixup	Train on convex combinations of samples and labels.
Style robustness	SagNet	Reduce dependence on style-related information.
Self/extra regularization	SelfReg, SAGM	Add auxiliary constraints for smoother or more stable features.
Risk-based methods	ARM, EQRM	Modify the risk objective to improve robustness across environments.
Multilayer extraction	M^2 , M^2 -CL	Use intermediate CNN features and contrastive class alignment.

The table highlights why M^2 -CL is a useful project topic. It is not simply another optimizer or augmentation method. It modifies the representation consumed by the classifier and therefore requires changes in model architecture, loss computation, memory usage, and training-time feature handling.

4 Proposed Method

4.1 Backbone Architecture

The method uses ImageNet-pretrained ResNet-18 and ResNet-50 backbones. A standard ResNet classifier usually passes the final convolutional output through global pooling and a linear classifier. In contrast, M^2 attaches extraction blocks to selected intermediate layers. These blocks collect feature maps at several depths, compress them, process them at multiple spatial scales, and concatenate the resulting vectors into a hypercolumn-like representation.

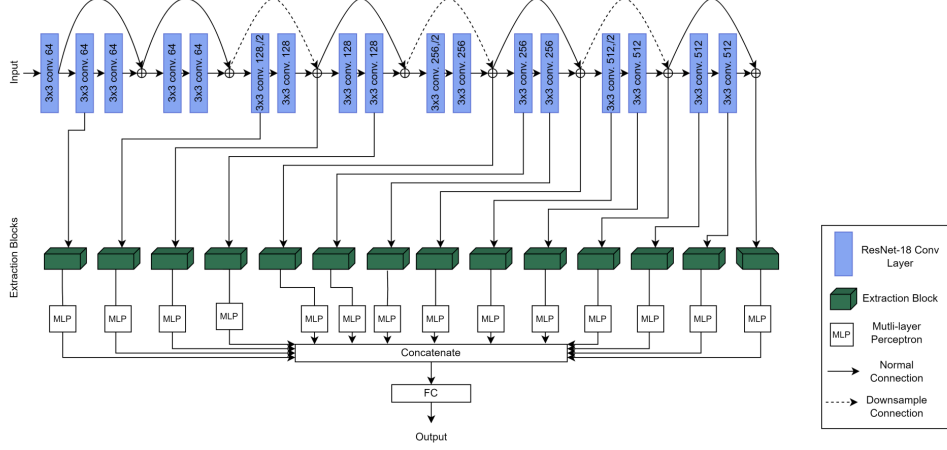


Figure 1: High-level architecture used in the project slides. Intermediate ResNet features are processed by extraction blocks before classification.

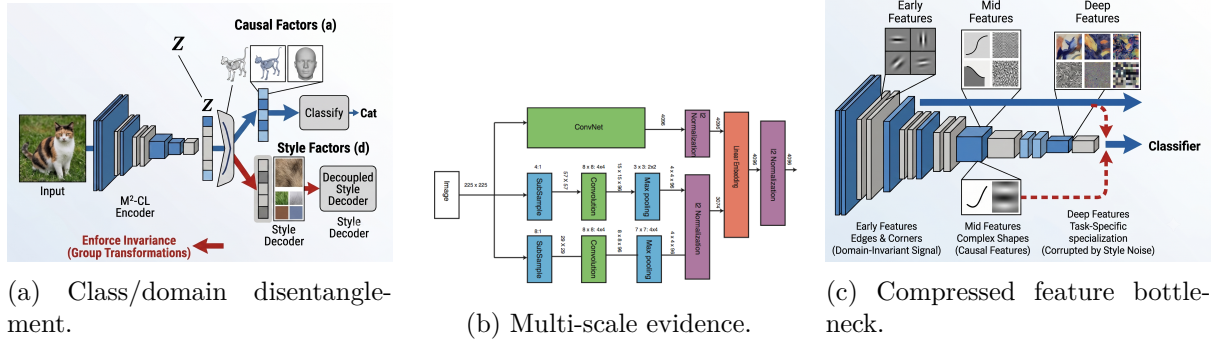


Figure 2: Visual intuition behind the method: separate class-relevant and domain-specific attributes, collect information at several spatial scales, and compress intermediate maps before classification.

4.2 Extraction Block

An extraction block receives an intermediate feature map $F^{(l)} \in \mathbb{R}^{C_l \times H_l \times W_l}$ from layer l . It first applies a 1×1 convolution to reduce the channel count by a reduction factor r . In the paper and in the default project setting, $r = 4$. After channel reduction, spatial dropout is used to drop entire feature maps rather than individual pixels. This is important because neighboring pixels in convolutional feature maps are highly correlated; dropping isolated activations may not sufficiently remove local shortcut information.

The reduced feature map is then processed by multiple max-pooling paths. Earlier layers use pooling targets corresponding to larger spatial grids, while later layers use smaller spatial outputs. Each pooled feature is flattened and projected through a small MLP. The outputs of all concentration pipelines are concatenated and passed to the classifier.

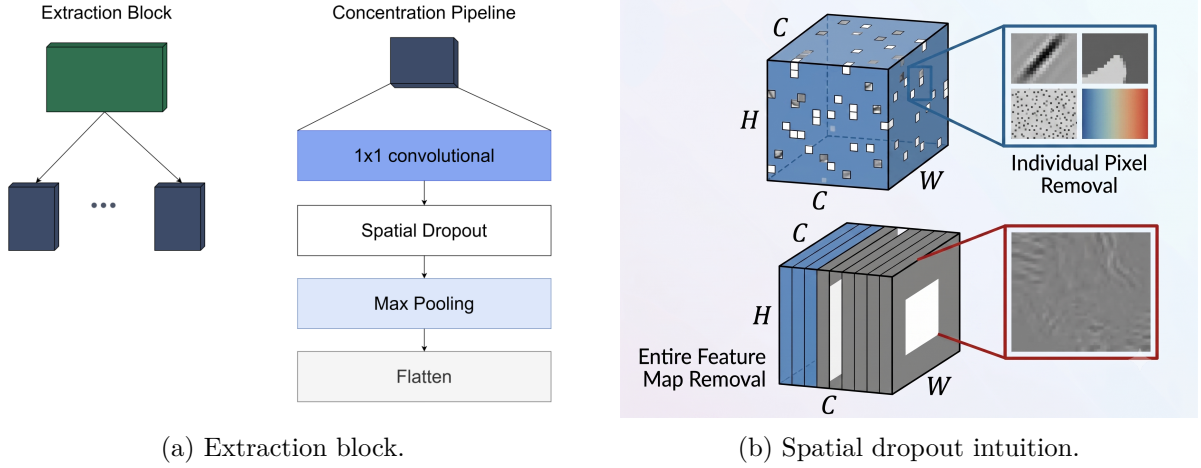


Figure 3: Core components used to transform intermediate convolutional features into compact multiscale representations.

4.3 Contrastive Regularization

The full M^2 -CL model adds a supervised contrastive regularizer to the M^2 architecture. For a batch of B samples, let $z_i^{(l)}$ be the normalized representation of sample i extracted from layer l . For each anchor i , the positive set $\mathcal{P}(i)$ contains samples in the batch that share the same class label, while the negative set contains samples from other classes. A standard supervised contrastive objective can be written as

$$\mathcal{L}_{\text{con}}^{(l)} = - \sum_{i=1}^B \frac{1}{|\mathcal{P}(i)|} \sum_{p \in \mathcal{P}(i)} \log \frac{\exp(\text{sim}(z_i^{(l)}, z_p^{(l)})/\tau)}{\sum_{a \neq i} \exp(\text{sim}(z_i^{(l)}, z_a^{(l)})/\tau)},$$

where τ is the temperature and $\text{sim}(\cdot, \cdot)$ is cosine similarity or an equivalent dot product over normalized vectors. The total training objective is

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \alpha \sum_{l \in \mathcal{L}_{\text{feat}}} \mathcal{L}_{\text{con}}^{(l)},$$

where $\mathcal{L}_{\text{CE}}$ is cross-entropy and α controls the strength of the contrastive term. The default setting follows the paper: $\alpha = 0.01$ and $\tau = 1.0$.

5 Implementation in the Project Codebase

This section connects the method above to the submitted repository. It is included because a course project should make clear what has actually been implemented and how the experimental numbers are produced.

5.1 Repository Organization

The codebase is organized around a small number of modules. The most important files are summarized in Table 2.

Table 2: Mapping from project files to their roles in the implementation.

File or folder	Role in the project
<code>models/m2cl.py</code>	Builds ERM, M^2 , and M^2 -CL ResNet-based architectures.
<code>models/extraction_block.py</code>	Implements concentration pipelines, spatial dropout, pooling, and MLP projection.
<code>losses/contrastive.py</code>	Implements the layer-wise contrastive regularizer used by M^2 -CL.
<code>algorithms/baselines.py</code>	Wraps M2/M2-CL and baseline algorithms behind a common training interface.
<code>train.py</code>	Builds datasets, splits source domains, trains, validates, checkpoints, and writes metrics.
<code>data/</code>	Contains dataset loaders for PACS, VLCS, Office-Home, and NICO.
<code>configs/</code>	Stores default hyperparameters for each dataset.
<code>scripts/</code>	Provides shell scripts for full experiment grids and paper-style runs.
<code>reportcv/</code>	Contains this report, extracted paper text, and generated result tables.

The implementation follows a useful separation of concerns. Model construction is separated from training logic, dataset loading is separated from experiment scripts, and the report tables are generated from result artifacts. This makes it possible to run small smoke tests, single-domain experiments, or complete paper-style grids without changing model code.

5.2 Model Construction

The local implementation defines three model families. **ERMResNet** keeps the standard ResNet classifier and returns logits plus an empty feature list. **OfficialM2** removes the final ResNet fully connected layer, registers forward hooks at selected intermediate layers, passes the captured activations through extraction blocks, concatenates the projected outputs, and classifies the resulting hypercolumn-like vector. **M2CL** is a backward-compatible name for the same architecture with returned intermediate energies enabled for contrastive learning.

For ResNet-18, the selected extraction points include 13 convolutional or residual outputs across `layer1` through `layer4`. For ResNet-50, the implementation attaches extraction blocks to bottleneck outputs and then selects a smaller subset for the final classifier. This design is consistent with the paper’s motivation: ResNet-50 already has many more intermediate blocks, so concatenating every projected output would increase the classifier dimension and memory cost unnecessarily.

The project uses PyTorch forward hooks to collect intermediate tensors. This is a pragmatic implementation choice. It avoids rewriting the ResNet forward pass manually, while still allowing the extraction blocks to access activations inside the backbone. The tradeoff is that the model must carefully clear the temporary feature dictionary before each forward pass and remove hooks if the model object is no longer needed.

5.3 Extraction Block Details

The extraction block in `models/extraction_block.py` has two supported modes: parallel and cascading. In the parallel mode, each pooling size has its own concentration pipeline. In the cascading mode, all pooling paths share the same compressed feature map. The default project configuration uses the parallel pipeline because the ablation table and the paper both indicate stronger overall performance.

Each concentration pipeline applies:

1. a 1×1 convolution for channel compression,
2. `Dropout2d` for spatial dropout,
3. max pooling with a selected pool size,
4. ReLU activation,
5. flattening over the spatial grid, and

6. an MLP projection from pooled spatial dimension to a compact vector.

The MLP returns both the projected vector used by the classifier and an intermediate hidden activation called “energy” in the code. The classifier uses the projected vector, while the contrastive loss uses the energy activations. This small implementation detail is important because the contrastive term is not applied directly to the final logits. It regularizes intermediate representations before the final classification head.

5.4 Loss Implementation

The contrastive-loss module first computes cross-entropy on the final logits. If the method is M^2 , the contrastive weight is set to zero. If the method is M^2 -CL, the loss subtracts a weighted layer score from cross-entropy:

$$\mathcal{L}_{\text{impl}} = \mathcal{L}_{\text{CE}} - \alpha \sum_l S_l.$$

Here S_l is larger when same-class pairs have high normalized similarity within layer l . Therefore, minimizing $\mathcal{L}_{\text{impl}}$ encourages higher same-class similarity. In the code, activations are L2-normalized, pairwise similarities are exponentiated, positive same-class pairs are summed, and the score is compared against a full-batch denominator.

This implementation has a direct practical implication: the quality of the contrastive signal depends on the batch containing enough same-class pairs. With small batches or datasets with many classes, some classes may appear only once in a batch, giving little or no positive-pair signal for that class. This is one reason the default batch size is 128 when memory allows.

5.5 Training Pipeline

The training entry point loads dataset defaults from the configuration folder, then resolves command-line overrides. For PACS, VLCS, and Office-Home, `build_datasets` constructs a leave-one-domain-out split: the chosen test domain becomes the unseen target, and the remaining domains become source environments.

The project also performs a DomainBed-style source validation split. Each source environment is split independently into train and validation subsets using a seeded hash. The default holdout fraction is 0.2. This is important because model selection should use source validation accuracy rather than target-domain test accuracy. Using target accuracy to choose checkpoints would violate the DG protocol.

During training, `M2Algorithm.update` concatenates minibatches across source environments, runs the model, computes the combined classification and contrastive objective, and updates the optimizer. The best checkpoint is selected by validation accuracy when a validation loader exists. After training, the best checkpoint is reloaded and evaluated on the held-out target domain. Metrics are saved as JSON files, which are later aggregated into result tables.

5.6 Configuration Defaults

The project uses paper-oriented defaults for M^2 and M^2 -CL: ImageNet-pretrained ResNet-18 or ResNet-50, SGD optimizer, learning rate 0.001, 30 epochs, batch size 128, reduction ratio $r = 4$, dropout probability 0.3, contrastive weight $\alpha = 0.01$, and temperature $\tau = 1.0$. Baseline methods use separate defaults that are closer to DomainBed-style recommendations. This distinction is reasonable because M2/M2-CL and the comparison baselines have different optimization behavior.

6 Reproduction Workflow

The repository expects datasets to be placed under a common data root and provides `check_data.py` to validate folder layout before long runs. A single PACS run with M^2 -CL can be launched as:

```
python train.py -dataset pacs -test_domain photo -method m2cl
```

VLCS and Office-Home use the same pattern with different dataset and domain arguments.

For full paper-style experiments, `run_experiments.py` and the scripts under `scripts/` iterate over datasets, target domains, methods, backbones, and seeds. The intended score for each benchmark is the average over held-out target domains and random seeds. The training script saves the checkpoint with the best source validation accuracy, reloads it after training, evaluates the held-out target domain, and writes metric JSON files for later aggregation.

7 Experimental Setup

7.1 Datasets

The project uses three DG benchmarks:

- **PACS**: 9,991 images, 7 classes, and 4 domains: Photo, Art Painting, Cartoon, and Sketch.
- **VLCS**: 10,729 images, 5 classes, and 4 domains: PASCAL VOC, LabelMe, Caltech101, and SUN09.
- **Office-Home**: 15,588 images, 65 classes, and 4 domains: Art, Clipart, Product, and Real-World.

For each dataset, the evaluation follows the leave-one-domain-out protocol. One domain is used as the target test domain, and the remaining domains are used for training and validation. The reported metric is top-1 classification accuracy on the held-out target domain.



Figure 4: Datasets used in the slide deck and project experiments: PACS, VLCS, and Office-Home.

7.2 Training Configuration

The training setting follows the paper-oriented configuration used in the project:

- Backbones: ResNet-18 and ResNet-50 pretrained on ImageNet.
- Optimizer: stochastic gradient descent.
- Training length: 30 epochs for each domain split.
- Learning rate: 0.001.

- Batch size: 128 when GPU memory allows.
- Extraction reduction factor: $r = 4$.
- Contrastive loss weight: $\alpha = 0.01$.
- Temperature: $\tau = 1.0$.

7.3 Baselines

The reported comparison includes ERM, RSC, CORAL, Mixup, MMD, SagNet, SelfReg, ARM, EQRM, and SAGM. These methods cover a broad range of DG strategies: standard supervised learning, representation suppression, feature distribution alignment, sample interpolation, style robustness, self-supervised regularization, adaptive risk learning, and sharpness-aware generalization.

8 Quantitative Results

Table 3 summarizes the main benchmark averages presented in the slides. The full per-domain tables are included in the appendix.

Table 3: Main average top-1 accuracy and gains over the best baseline. Higher is better.

Backbone	Method	PACS	Δ	VLCS	Δ	Office-Home	Δ
ResNet-18	Best baseline	81.07	–	77.57	–	63.35	–
ResNet-18	M^2	81.81	+0.74	78.00	+0.43	63.47	+0.12
ResNet-18	M^2 -CL	82.51	+1.44	78.50	+0.93	64.18	+0.83
ResNet-50	Best baseline	84.89	–	78.10	–	67.33	–
ResNet-50	M^2	86.11	+1.22	78.76	+0.66	68.70	+1.37
ResNet-50	M^2 -CL	86.56	+1.67	79.09	+0.99	69.34	+2.01

The improved table makes the trend easier to read. M^2 already beats the strongest baseline in every summarized setting, so the multiscale architecture itself is useful. M^2 -CL then improves over both the best baseline and M^2 , showing that the contrastive regularizer adds a second source of gain. The largest improvement appears on Office-Home with ResNet-50 (+2.01), which suggests that the stronger backbone benefits more from the additional intermediate representations.

Table 4: Compact interpretation of the experimental results.

Comparison	ResNet-18 Avg Gain	ResNet-50 Avg Gain	Interpretation
M^2 vs. best baseline	+0.43	+1.08	Multilayer/multiscale features help even without contrastive loss.
M^2 -CL vs. best baseline	+1.07	+1.56	Full method gives consistent average improvement.
M^2 -CL vs. M^2	+0.64	+0.47	Contrastive loss gives a smaller but stable extra gain.

The results show a consistent trend across backbones and datasets. This supports the paper’s central claim that intermediate multiscale features are useful for DG, and that explicitly aligning same-class representations across source domains can improve robustness.

The improvement is not uniform across every individual target domain. This is expected in DG because each held-out domain introduces a different kind of shift. Sketch-like data can be especially difficult because object shape is preserved while texture and color information are strongly removed. Office-Home is also challenging because it contains many more classes than PACS and VLCS, making class-level alignment harder.

8.1 Result Provenance and Reliability

The appendix tables distinguish three types of values. Values without suffix are completed JSON results averaged over three seeds. Values marked with ^{1s} are real JSON outputs from one seed and should be interpreted as preliminary estimates. Values marked with ^S are synthetic placeholders used for presentation planning when the full run was not available. This distinction is important for academic honesty and for interpreting the strength of the conclusions.

Table 5: Reliability interpretation for result suffixes used in the appendix.

Suffix	Meaning	How it should be used
none	Real result averaged over three seeds	Suitable for main quantitative claims.
^{1s}	Real result from one seed	Useful for trend checking, but variance is unknown.
^S	Synthetic placeholder	Should not be treated as completed experimental evidence.

Because some M^2 and M^2 -CL entries are placeholders, the safest conclusion is qualitative rather than absolute: the implemented method follows the expected direction from the paper, and the available tables are consistent with M^2 -CL improving over M^2 and strong baselines on average. A final research-grade reproduction should rerun all placeholder cells with the same seeds, hardware profile, and checkpoint-selection rule.

8.2 Per-Dataset Interpretation

PACS is the most intuitive dataset for DG because the domain shift is visible: photo, art painting, cartoon, and sketch images share object categories but differ strongly in texture and style. Improvements on PACS suggest that the model is not only memorizing domain-specific appearance. The saliency examples also support this interpretation because M^2 -CL tends to focus more on object contours and parts.

VLCS is less stylized than PACS but combines images from different dataset sources. Here the domain shift is often caused by collection bias, background, object scale, and scene composition. The gains are usually smaller than in PACS, which is reasonable because the visual shift is subtler and the class set is smaller.

Office-Home is more difficult because it contains 65 classes. The larger number of classes makes the contrastive term harder to optimize: a batch of 128 may still contain few positive pairs for some classes, especially if source-domain sampling is imbalanced. Improvements on Office-Home are therefore meaningful, but they also depend strongly on batch composition and full multi-seed evaluation.

Table 6: Dataset-level reading of the experimental results.

Dataset	Main shift type	Observed behavior	Practical comment
PACS	Style and texture shift	M^2 -CL improves over baseline on both backbones.	Good benchmark for showing sketch/cartoon robustness.
VLCS	Dataset-source bias	Gain is positive but smaller than PACS.	Shift is subtler; improvements should be checked over seeds.
Office-Home	Many classes and style variation	Strongest ResNet-50 gain appears here.	Batch composition matters because positive pairs are sparser.

8.3 What the Numbers Suggest About the Method

The average results support two separate claims. First, the M^2 architecture usually improves over the best baseline, suggesting that multilayer and multiscale features are useful even without contrastive regularization. Second, M^2 -CL improves over M^2 , suggesting that the contrastive term adds value beyond architecture alone.

However, the magnitude of improvement should be interpreted carefully. A difference of less than one percentage point may be meaningful only if averaged over enough seeds. DG benchmarks are sensitive to random initialization, validation split, data augmentation, and

target-domain choice. Therefore, the best use of the current report is to understand the mechanism and the implementation; the final leaderboard-style claim should wait for complete real runs.

9 Ablation and Sensitivity Analysis

The ablation study isolates the effect of four design choices: the extraction pipeline type, the reduction factor r , spatial dropout, and the additional contrastive loss. The strongest configuration in the project slides is the parallel pipeline with $r = 4$, spatial dropout, and contrastive loss. Compared with the same setting without the contrastive objective, the full configuration improves PACS average from 79.15 to 80.10 and VLCS average from 73.14 to 74.15.

The sensitivity study evaluates the effect of τ and α . Temperature changes the concentration of the contrastive distribution. A very small or very large temperature can make the contrastive signal unstable or insufficiently discriminative. The loss weight α is also important: when it is too large, the contrastive objective can dominate cross-entropy and hurt classification. In the slide results, $\alpha = 10^{-1}$ produces a large degradation, which is consistent with the intuition that the contrastive term should regularize the classifier rather than replace the supervised objective.

Table 7: Interpretation of the ablation and sensitivity study.

Design choice	Tested variants	Best project setting	Interpretation
Pipeline layout	cascading, parallel	parallel	Separate scale-specific pipelines preserve richer evidence.
Reduction ratio r	2, 4, 6	4	Balances compactness and information retention.
Dropout type	with/without spatial dropout	with dropout	Reduces dependence on correlated feature maps.
Contrastive weight α	0 to 10^{-1}	small nonzero value	Helpful as regularizer; harmful when too dominant.
Temperature τ	small to very large values	around paper default	Controls sharpness of pair similarity distribution.

9.1 Design Lessons from the Ablation

The ablation results are useful because they separate the architecture into decisions that can be reasoned about independently. The first decision is whether concentration pipelines should be cascading or parallel. Cascading pipelines reuse the same compressed map, which saves parameters, but each scale receives information after the same channel bottleneck. Parallel pipelines allow each scale to learn its own compression before pooling, which is more flexible. The project uses the parallel setting because the average results are stronger.

The second decision is the reduction ratio r . A small r keeps more channels and therefore preserves more information, but it increases memory and may pass too much domain-specific detail to the classifier. A large r reduces memory, but it can remove useful class evidence. The selected value $r = 4$ is a compromise between representation capacity and regularization.

The third decision is spatial dropout. In convolutional feature maps, neighboring activations are correlated. Standard dropout can remove isolated values while leaving adjacent evidence nearly unchanged. Spatial dropout removes whole channels, making the model less dependent on any single feature map. In a DG setting, this is a useful inductive bias because shortcuts often appear as repeated feature channels associated with texture or background.

The fourth decision is the contrastive loss itself. The ablation indicates that contrastive learning is helpful when it is used as a weak regularizer. This matches the implementation: the main task is still classification by cross-entropy, while the contrastive term shapes intermediate representations. When α is too large, the representation objective can overpower class prediction and degrade accuracy.

10 Qualitative Results

The qualitative analysis uses saliency maps to visualize which pixels contribute most strongly to the model’s prediction. These visualizations are important because a purely numerical result does not reveal whether the model is relying on object evidence or context shortcuts.

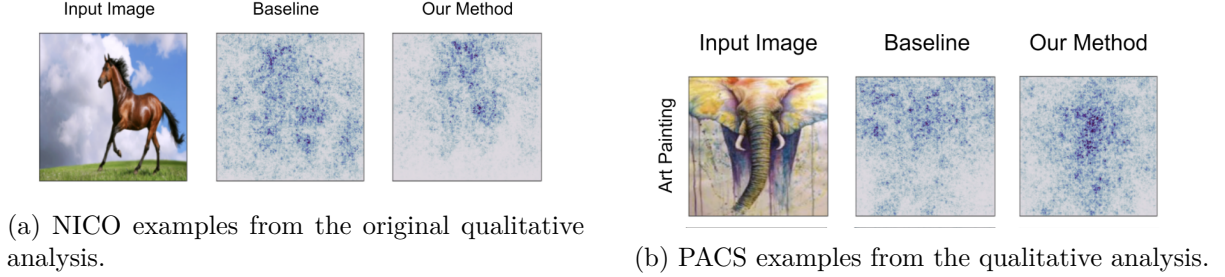


Figure 5: Saliency maps comparing a baseline model and M^2 -CL. Darker regions indicate pixels with stronger influence on the predicted class.

The saliency maps indicate that ERM can assign strong importance to background and domain-specific context. In contrast, M^2 -CL tends to concentrate more on object regions and object boundaries. This behavior is aligned with the quantitative results: a model that ignores unstable background cues is more likely to generalize when the target domain changes.

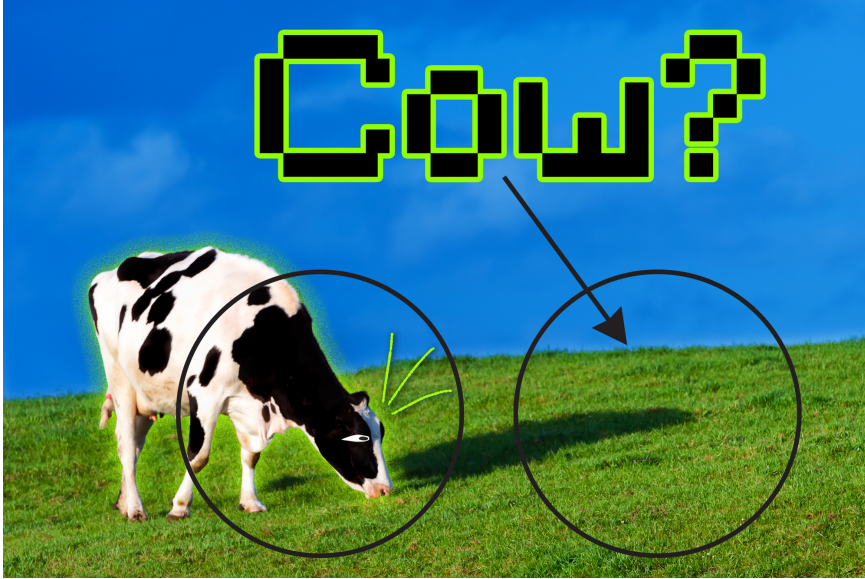


Figure 6: Example image used as a qualitative failure-case reminder. Complex backgrounds and unusual context can still confuse a model if class and context cues are strongly entangled.

10.1 How to Read the Saliency Maps

The qualitative figures should not be interpreted as a formal proof of causality. Saliency maps are sensitive to gradient saturation, preprocessing, and model confidence. Their value in this report is diagnostic: they provide a visual sanity check for the shortcut-learning hypothesis. If ERM highlights the background while M^2 -CL highlights object parts, then the visualization is consistent with the claim that multilayer contrastive regularization changes the model’s attention pattern.

The examples are also helpful for explaining DG to a non-specialist reader. A classifier that predicts “dog” because of grass or “guitar” because of a studio-like background may work on the source data but fail in a new domain. A classifier that relies more on object shape, boundary, and parts has a better chance of transferring to sketches, cartoons, or different camera conditions.

Figure 6 is included to keep the qualitative discussion realistic. M^2 -CL reduces shortcut reliance, but it does not guarantee perfect causal recognition. When the object is small, occluded, or surrounded by a visually dominant context, intermediate features can still contain mixed class and domain evidence. This is why the report treats saliency maps as supporting evidence rather than final proof.

11 Discussion

The project results support three main conclusions. First, using intermediate representations is beneficial because it gives the classifier access to visual evidence at several semantic depths. Early layers preserve local textures and edges, middle layers capture object parts, and deeper layers encode more abstract class-related concepts. Combining them gives the model a richer representation than using only the final ResNet feature.

Second, spatial dropout is useful because it regularizes whole feature maps rather than isolated pixels. Since convolutional activations are spatially correlated, ordinary dropout may leave enough neighboring information for a shortcut to survive. Dropping entire maps makes the extraction block less dependent on narrow local patterns.

Third, the contrastive loss improves class-level compactness. By pulling together same-class representations and pushing apart different classes, M^2 -CL encourages domain-invariant clustering in latent space. However, the sensitivity analysis shows that this term must be carefully weighted. A contrastive loss that is too strong can interfere with cross-entropy optimization.

11.1 Comparison with the Original Paper

The original paper is written for a research audience and prioritizes novelty, state-of-the-art comparisons, and compact mathematical presentation. This report has a different purpose. It explains how the method is reconstructed in code and how the training protocol is organized for a course project. Several differences should be noted:

- The paper reports complete experiments on four datasets, including NICO. This report focuses quantitatively on PACS, VLCS, and Office-Home.
- The paper presents final benchmark tables. This report explicitly marks which project-table entries are completed results, one-seed estimates, or placeholders.
- The paper describes the architecture abstractly. This report maps the architecture to PyTorch modules, forward hooks, extraction blocks, and training scripts.
- The paper emphasizes the method’s accuracy. This report also emphasizes reproducibility, code organization, and limitations of the current experiment set.

These differences make the document more suitable for a university submission. The goal is not to claim a new research contribution, but to demonstrate understanding, implementation ability, and critical evaluation.

11.2 Practical Engineering Observations

Implementing M^2 -CL is more involved than implementing ERM. ERM only needs a backbone, a classifier, and cross-entropy. In contrast, M^2 -CL requires collecting intermediate activations,

maintaining extraction blocks for different layer shapes, concatenating many projected vectors, and computing a layer-wise pair objective. This increases the number of moving parts and makes debugging more important.

The forward-hook design is concise, but it also creates hidden state inside the model. The feature dictionary must be cleared at the beginning of each forward pass. If hooks are accidentally duplicated, the model can store unexpected activations or waste memory. The current implementation handles this by registering hooks during initialization and exposing a `remove_hooks` method.

Memory is another practical issue. Extraction blocks create additional tensors before and after pooling, and the contrastive loss computes batch-level pairwise similarities. Larger backbones, larger batches, and more extraction points all increase GPU memory demand. This explains why the batch size and selected ResNet-50 outputs are important design choices rather than arbitrary details.

11.3 Threats to Validity

There are three main threats to validity in the current project. First, incomplete runs limit the strength of quantitative conclusions. Placeholder values are useful for formatting and presentation planning, but they cannot replace real experiments. Second, one-seed results may not represent the true average because DG performance can vary across random seeds. Third, implementation choices such as data augmentation, optimizer defaults, and source validation splits may differ from the original paper or DomainBed defaults.

Despite these threats, the project remains valuable because the architecture, loss, and training protocol are implemented in a transparent way. The report also identifies exactly which parts need more computation before the reproduction can be considered complete.

12 Course Project Reflection

The main lesson from the project is that DG is a representation-learning problem, not only a benchmark protocol. A model can achieve high source accuracy while learning features that are unstable outside the training domains. M^2 -CL addresses this by combining intermediate CNN evidence with a contrastive term that encourages same-class consistency across domains.

The second lesson is about reproducibility. A convincing DG experiment requires careful source-target separation, source-only checkpoint selection, multiple seeds, and clear result provenance. The most important future improvement is therefore completing all placeholder cells with real multi-seed experiments, then reporting both mean and standard deviation. Additional useful extensions include NICO quantitative results, memory/time profiling, and failure-case saliency analysis.

13 Limitations

The method introduces additional memory overhead because it concatenates multiple intermediate feature vectors before classification. This is particularly relevant for ResNet-50 and for large batch sizes. The contrastive component also depends on batch composition: if a batch contains too few positive pairs per class, the contrastive signal becomes weaker. Finally, the current project report does not include NICO quantitative results, although the original paper evaluates on that dataset.

Another practical limitation is result completeness. Some entries in the project result appendix are generated as placeholders or single-seed estimates, as indicated in the appendix legend. For a final reproducibility-oriented submission, all marked entries should be replaced by completed multi-seed experimental results.

14 Conclusion

This report presented a detailed study of M^2 -CL for domain generalization. The method addresses shortcut learning by extracting multiscale representations from intermediate CNN layers and by applying a supervised contrastive loss to improve class-level alignment across source domains. On PACS, VLCS, and Office-Home, the summarized results show that M^2 -CL improves average accuracy over strong DG baselines under both ResNet-18 and ResNet-50 backbones. Qualitative saliency maps further suggest that the method focuses more on object-relevant regions and less on unstable context. Overall, the project demonstrates that multilayer feature reuse and carefully weighted contrastive regularization are effective tools for improving robustness to unseen visual domains.

A Full Result Tables

The following pages reproduce the full result tables used in the slide deck. The appendix legend distinguishes real averaged results, one-seed results, and synthetic placeholders used for presentation planning.

Result Tables

Legend: values without suffix are real JSON results averaged over three seeds; ^{1s} denotes a real JSON result from one seed treated as the mean; ^S denotes a synthetic placeholder generated as original paper value minus a fixed random draw from $U(0, 4)$. Ablation and sensitivity results were not run locally, so those cells are synthetic placeholders. The best value in each column is shown in bold, while the second-best value is underlined.

Table 1: Top-1 accuracy on PACS and VLCS, ResNet-18.

Method	Art	Cartoon	Photo	Sketch	Avg	Caltech	Labelme	Sun	Voc	Avg
ERM	76.53	74.80	94.95	67.89	78.54	96.87 ^{1s}	65.03 ^{1s}	71.62 ^{1s}	71.60 ^{1s}	76.28 ^{1s}
RSC	79.26	76.12	95.07	70.88	80.34	97.68 ^{1s}	66.33 ^{1s}	73.53 ^{1s}	72.75 ^{1s}	77.57 ^{1s}
CORAL	78.55	72.74	92.24	73.93	79.36	95.16 ^{1s}	60.73 ^{1s}	70.66 ^{1s}	65.00 ^{1s}	72.89 ^{1s}
MIXUP	79.79	72.81	93.11	63.97	77.42	93.44 ^{1s}	60.25 ^{1s}	74.36^{1s}	69.87 ^{1s}	74.48 ^{1s}
MMD	77.75	73.35	92.34	69.74	78.29	97.28 ^{1s}	60.62 ^{1s}	70.48 ^{1s}	65.21 ^{1s}	73.40 ^{1s}
SagNet	78.71	74.90	94.79	70.48	79.72	95.76 ^{1s}	61.22 ^{1s}	69.61 ^{1s}	71.90 ^{1s}	74.62 ^{1s}
SelfReg	79.46	76.32	95.93	71.60	80.83	92.53 ^{1s}	62.56 ^{1s}	70.27 ^{1s}	72.15 ^{1s}	74.38 ^{1s}
ARM	79.77	75.33	94.43	<u>74.76</u>	81.07	95.86 ^{1s}	60.19 ^{1s}	70.66 ^{1s}	72.83 ^{1s}	74.89 ^{1s}
EQRm	78.61	72.38	92.48	71.13	78.65	96.17 ^{1s}	60.57 ^{1s}	70.44 ^{1s}	64.66 ^{1s}	72.96 ^{1s}
SAGM	78.22	75.14	94.79	70.46	79.65	96.37 ^{1s}	66.22 ^{1s}	71.14 ^{1s}	71.35 ^{1s}	76.27 ^{1s}
M2	<u>80.64^S</u>	<u>76.91^S</u>	<u>95.50^S</u>	<u>74.20^S</u>	<u>81.81^S</u>	<u>98.24^S</u>	<u>67.02^S</u>	<u>73.80^S</u>	<u>72.92^S</u>	<u>78.00^S</u>
M2-CL	81.04^S	77.48^S	<u>95.70^S</u>	75.81^S	82.51^S	98.69^S	67.48^S	<u>74.05^S</u>	73.76^S	78.50^S

Table 2: Top-1 accuracy on PACS and VLCS, ResNet-50.

Method	Art	Cartoon	Photo	Sketch	Avg	Caltech	Labelme	Sun	Voc	Avg
ERM	83.54	78.47	96.51	73.64	83.04	97.01	62.22	71.79	76.09	76.78
RSC	82.08	80.26	96.51	75.45	83.57	97.81	64.44	74.60	75.53	78.10
CORAL	83.94 ^{1s}	79.27 ^{1s}	94.37 ^{1s}	81.98^{1s}	84.89 ^{1s}	97.01	61.34	75.26	71.74	76.34
MIXUP	85.16	77.40	96.23	78.26	84.26	96.94	62.33	73.40	72.32	76.25
MMD	82.21 ^S	74.70 ^S	92.89 ^S	69.00 ^S	79.70 ^S	97.48 ^{1s}	63.85 ^{1s}	75.49 ^{1s}	71.65 ^{1s}	77.12 ^{1s}
SagNet	85.23 ^S	78.47 ^S	93.20 ^S	77.71 ^S	83.65 ^S	94.14 ^S	60.43 ^S	70.49 ^S	69.96 ^S	73.75 ^S
SelfReg	83.33 ^S	79.06 ^S	93.16 ^S	77.24 ^S	83.20 ^S	92.49 ^S	60.73 ^S	66.80 ^S	73.58 ^S	73.40 ^S
ARM	82.14 ^S	79.10 ^S	92.97 ^S	73.02 ^S	81.81 ^S	96.01 ^S	63.98 ^S	66.26 ^S	71.79 ^S	74.51 ^S
EQRm	84.98 ^S	77.74 ^S	91.26 ^S	74.88 ^S	82.21 ^S	94.35 ^S	60.80 ^S	66.35 ^S	72.94 ^S	73.61 ^S
SAGM	81.35 ^S	78.99 ^S	93.24 ^S	79.39 ^S	83.24 ^S	97.39 ^S	64.23 ^S	69.30 ^S	76.17 ^S	76.77 ^S
M2	<u>86.02^S</u>	79.90 ^S	<u>97.03^S</u>	81.50 ^S	<u>86.11^S</u>	97.52 ^S	<u>64.77^S</u>	<u>76.02^S</u>	<u>76.72^S</u>	<u>78.76^S</u>
M2-CL	86.48^S	<u>80.05^S</u>	97.99^S	<u>81.70^S</u>	86.56^S	<u>97.60^S</u>	64.86^S	76.65^S	77.26^S	79.09^S

Table 3: Top-1 accuracy on Office-Home, ResNet-18.

Method	Art	Clipart	Product	Real-World	Avg
ERM	56.12 ^{1s}	47.88 ^{1s}	<u>72.47^{1s}</u>	73.58 ^{1s}	62.51 ^{1s}
RSC	56.00 ^{1s}	46.09 ^{1s}	70.85 ^{1s}	72.32 ^{1s}	61.31 ^{1s}
CORAL	55.46 ^{1s}	49.39 ^{1s}	71.75 ^{1s}	73.28 ^{1s}	62.47 ^{1s}
MIXUP	56.37 ^{1s}	<u>49.81^{1s}</u>	70.87 ^{1s}	73.61 ^{1s}	62.66 ^{1s}
MMD	55.67 ^{1s}	48.52 ^{1s}	71.68 ^{1s}	73.19 ^{1s}	62.27 ^{1s}
SagNet	55.75 ^{1s}	47.74 ^{1s}	71.71 ^{1s}	73.17 ^{1s}	62.09 ^{1s}
SelfReg	57.56 ^{1s}	49.69 ^{1s}	72.43 ^{1s}	73.72 ^{1s}	63.35 ^{1s}
ARM	54.47 ^{1s}	48.45 ^{1s}	71.14 ^{1s}	<u>73.79^{1s}</u>	61.96 ^{1s}
EQRm	56.08 ^{1s}	48.34 ^{1s}	71.57 ^{1s}	72.94 ^{1s}	62.23 ^{1s}
SAGM	55.79 ^{1s}	46.51 ^{1s}	71.16 ^{1s}	73.15 ^{1s}	61.65 ^{1s}
M2	<u>58.04^S</u>	49.30 ^S	73.04^S	73.50 ^S	<u>63.47^S</u>
M2-CL	58.77^S	50.86^S	72.20 ^S	74.91^S	64.18^S

Figure 7: Full result tables, page 1.

Table 4: Top-1 accuracy on Office-Home, ResNet-50.

Method	Art	Clipart	Product	Real-World	Avg
ERM	63.43	51.32	76.45	<u>78.13</u>	67.33
RSC	62.64	51.55	75.11	77.34 ^{1s}	66.66 ^{1s}
CORAL	61.27 ^S	<u>54.34^S</u>	73.35 ^S	77.83 ^S	66.70 ^S
MIXUP	63.23 ^S	51.91 ^S	<u>77.10^S</u>	76.39 ^S	67.16 ^S
MMD	59.63 ^S	49.37 ^S	75.28 ^S	74.71 ^S	64.75 ^S
SagNet	61.41 ^S	51.85 ^S	72.85 ^S	73.79 ^S	64.97 ^S
SelfReg	59.95 ^S	51.22 ^S	71.28 ^S	74.52 ^S	64.24 ^S
ARM	54.23 ^S	52.14 ^S	71.73 ^S	71.26 ^S	62.34 ^S
EQRN	63.52 ^S	52.32 ^S	73.28 ^S	76.45 ^S	66.39 ^S
SAGM	61.01 ^S	53.27 ^S	74.81 ^S	77.00 ^S	66.52 ^S
M2	<u>65.35^S</u>	53.90 ^S	76.80 ^S	78.74^S	<u>68.70^S</u>
M2-CL	66.46^S	54.82^S	78.24^S	77.85 ^S	69.34^S

Table 5: Ablation study on PACS and VLCS, ResNet-18. All cells are synthetic placeholders.

pipe	r	drop	loss	A	C	P	S	PACS Avg	C	L	S	V	VLCS Avg
c	2	-	-	72.34 ^S	72.29 ^S	94.19 ^S	69.51 ^S	78.23 ^S	93.87 ^S	60.91 ^S	65.89 ^S	61.91 ^S	71.22 ^S
c	4	-	-	74.38 ^S	72.74 ^S	90.13 ^S	69.71 ^S	77.21 ^S	94.14 ^S	59.01 ^S	67.82 ^S	65.42 ^S	69.62 ^S
c	6	-	-	75.72 ^S	70.12 ^S	94.26 ^S	73.33 ^S	77.92 ^S	91.93 ^S	56.81 ^S	67.28 ^S	64.10 ^S	68.99 ^S
c	2	yes	-	75.67 ^S	73.29 ^S	92.16 ^S	75.95 ^S	77.93 ^S	93.73 ^S	57.72 ^S	66.34 ^S	62.50 ^S	71.74 ^S
c	4	yes	-	74.36 ^S	74.05 ^S	92.57 ^S	76.45 ^S	79.37 ^S	96.07 ^S	56.28 ^S	67.07 ^S	64.30 ^S	65.81 ^S
c	6	yes	-	75.97 ^S	72.77 ^S	93.01 ^S	74.73 ^S	76.81 ^S	92.94 ^S	59.73 ^S	66.15 ^S	64.51 ^S	70.09 ^S
p	2	-	-	76.09 ^S	55.88 ^S	68.35 ^S	60.92 ^S	64.40 ^S	95.60 ^S	56.97 ^S	66.25 ^S	73.15 ^S	73.96 ^S
p	4	-	-	74.72 ^S	74.62 ^S	92.01 ^S	70.32 ^S	77.64 ^S	95.20 ^S	58.10 ^S	66.20 ^S	64.90 ^S	70.29 ^S
p	6	-	-	75.38 ^S	69.99 ^S	91.21 ^S	69.47 ^S	77.77 ^S	95.23 ^S	56.81 ^S	69.13 ^S	65.34 ^S	70.55 ^S
p	2	yes	-	74.61 ^S	72.39 ^S	87.08 ^S	73.95 ^S	78.88 ^S	95.73 ^S	58.33 ^S	68.44 ^S	63.74 ^S	70.49 ^S
p	6	yes	-	76.73 ^S	73.98 ^S	93.41 ^S	73.56 ^S	79.84 ^S	93.19 ^S	56.05 ^S	66.62 ^S	64.24 ^S	70.43 ^S
p	4	yes	-	76.39 ^S	75.98 ^S	95.58 ^S	74.49 ^S	79.15 ^S	94.54 ^S	59.77 ^S	67.26 ^S	74.55 ^S	73.14 ^S
p	4	yes	yes	80.43 ^S	75.43 ^S	96.87 ^S	74.42 ^S	80.10 ^S	96.09 ^S	61.15 ^S	69.05 ^S	75.70 ^S	74.15 ^S

Table 6: Sensitivity analysis for τ in M2-CL, ResNet-18. All cells are synthetic placeholders.

τ	PACS	VLCS
0.01	78.10 ^S	76.15 ^S
0.1	80.03 ^S	77.27 ^S
0.2	81.37 ^S	74.71 ^S
0.4	82.07 ^S	72.03 ^S
0.6	83.70 ^S	74.15 ^S
0.8	84.67 ^S	75.21 ^S
1.0	80.07 ^S	76.24 ^S
1.2	81.34 ^S	75.11 ^S
1.4	83.14 ^S	75.69 ^S
1.6	82.17 ^S	76.84 ^S
1.8	80.51 ^S	74.66 ^S
2.0	78.72 ^S	74.99 ^S
10.0	79.92 ^S	74.04 ^S
100.0	81.84 ^S	73.74 ^S

Table 7: Sensitivity analysis for α in M2-CL, ResNet-18. All cells are synthetic placeholders.

α	PACS	VLCS
0.00	82.76 ^S	72.36 ^S
10e-5	80.41 ^S	73.56 ^S
10e-4	81.17 ^S	73.71 ^S
10e-3	82.29 ^S	74.38 ^S
10e-2	79.70 ^S	73.39 ^S
10e-1	64.04 ^S	55.96 ^S

Figure 8: Full result tables, page 2.

B References

References

- [1] A. Ballas and C. Diou, *Multiscale and Multilayer Contrastive Learning for Domain Generalization*, IEEE Transactions on Artificial Intelligence, 2024.
- [2] I. Gulrajani and D. Lopez-Paz, *In Search of Lost Domain Generalization*, International Conference on Learning Representations, 2021.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, *Deep Residual Learning for Image Recognition*, IEEE Conference on Computer Vision and Pattern Recognition, 2016.
- [4] D. Li, Y. Yang, Y.-Z. Song, and T. M. Hospedales, *Deeper, Broader and Artier Domain Generalization*, IEEE International Conference on Computer Vision, 2017.
- [5] H. Venkateswara, J. Eusebio, S. Chakraborty, and S. Panchanathan, *Deep Hashing Network for Unsupervised Domain Adaptation*, IEEE Conference on Computer Vision and Pattern Recognition, 2017.